SYRAX — Project Reference & Viva FAQ

Source-code guide, system information and likely viva questions with model answers

Team: Ayush Kumar Upadhyay (2500350100012) · Navam (2500350100041) · Sandeep (2500350100057) · October 2026

A. Project Overview

Syrax is a voice-driven web agent with a 3D particle interface. A single spoken sentence is either executed as a real browser action (open, search, play, close) or answered as conversation (identity, time, date, world clock, general questions). The two paths never mix: actions come only from a deterministic local command parser, and the large language model can only talk — it has no way to open or navigate a tab.

Table A1: Quick facts

Item	Detail
Live site	https://ayushupadhyay90.github.io/Syrax/ (one responsive PWA URL)
Source repository	https://github.com/ayushupadhyay90/Syrax (public, key-free)
Project type	Web application / PWA with AI (NLP) and voice interaction
Frontend	React 19 + TypeScript 5.7 + Vite 6 + Tailwind CSS 4
3D / animation	Three.js 0.174, React Three Fiber 9, Drei 10, GSAP 3.12
Speech	Web Speech API (STT), layered TTS chain (Fish → Pocket → browser voice)
Language model	DeepSeek deepseek-flash — JSON mode, temperature 0.5, max_tokens 384
Hosting / CI	GitHub Actions → GitHub Pages (push to main branch)
Sidecar (optional)	Python FastAPI service (localhost) for premium voice features
Storage	Browser localStorage only — keys: syrax-chat-v1, syrax-ds-key. No database.
Cost	Near zero — free hosting, free browser speech, optional pay-per-use LLM key

B. Source Code & Repository

Statistics: ~3,887 lines across 14 TypeScript/TSX/CSS source files, plus ~310 lines of Python in the optional backend. Largest modules: chunks/Stage.tsx (1,811 lines — the whole interface and control logic) and ai/syrax.ts (505 lines — intent parser + LLM).

```
Syrax/                              GitHub repository root
|-- .github/workflows/deploy.yml    CI: builds the app, deploys to GitHub Pages
```

```
|-- presentation/                    Project decks + python-pptx generators
|-- docs/                            This synopsis and reference material
`-- syrax/                           The web application
    |-- package.json                 scripts: dev, build (tsc -b && vite build), preview,
server
    |-- backend/                     Optional FastAPI sidecar
    |   |-- main.py                  240 lines - proxy for premium voice services
    |   |-- setup_voice.py           voice setup helper
    |   `-- requirements.txt, .env.example
    `-- src/
        |-- main.tsx, App.tsx        entry + routing shell
        |-- chunks/Stage.tsx         1,811 lines - HUD, mic control, chat, command
execution
        |-- chunks/Chunk1.tsx        side operation panel
        |-- ai/syrax.ts              505 lines - local intent gate, LLM caller, prompt,
identity/time
        |-- ai/about.ts              creator credits + About panel copy
        |-- voice/stt.ts             speech recognizer wrapper (buffering, language en-IN)
        |-- voice/tts.ts             TTS chain: Fish -> Pocket -> browser speechSynthesis
        |-- voice/level.ts           microphone level helper for the particles
        |-- three/Background.tsx     canvas host / state binding
        |-- three/particles.ts       particle engine (state + mic-driven intensity)
        |-- browser/BrowserPanel.tsx  in-app site panel (secondary; real tabs are primary)
        `-- styles.css               380 lines - theme (blue #3B82F6 on #040711) and
layout
```

Build and run:

- **`$ npm install`** — install dependencies (in the syrax/ folder)
- **`$ npm run dev`** — local development server with hot reload
- **`$ npm run build`** — type-check (tsc -b) then production Vite build
- **`$ python -m uvicorn main:app --port 8000`** — optional: local voice sidecar (backend/)
- **`$ git push origin main`** — GitHub Actions builds and republishes the live site

Module responsibility map

Table B1: Modules and responsibilities

Module	Responsibility	Lines
chunks/Stage.tsx	Microphone lifecycle (one-shot), STT buffering/flush, chat UI, command execution, tab management, halt, song-prompt state	1,811
ai/syrax.ts	Opener/wake-word stripping, local intent gate (open/search/play/close), DeepSeek caller with JSON-mode prompt, identity + world-clock local answers	505
voice/stt.ts	Web Speech recognizer wrapper: continuous listening, interim results, lazy construction so it works even if injected late	189
voice/tts.ts	Speech output chain with timeouts and graceful degradation (Fish Audio → Pocket TTS → browser voice)	142
browser/BrowserPanel.tsx	Embedded site panel used as a secondary view; primary operations always open real	321

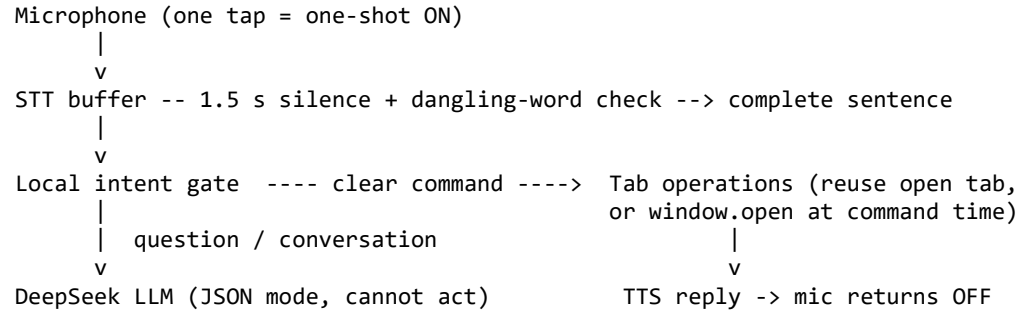
	Chrome tabs	
three/particles.ts + Background.tsx	Three.js particle field; intensity driven by app state (idle/listening/thinking/speaking) and microphone level	142
ai/about.ts	Canonical creator credits (Ayush, Navam, Sandeep) and About panel	36
backend/main.py	Optional FastAPI sidecar proxying premium voice services	240

Persistent data (the project uses no database)

Table B2: localStorage keys

Key	Purpose
syrax-chat-v1	Conversation history — restored on every page load; also seeds the LLM memory (last ~18 turns, JSON-shaped)
syrax-ds-key	User's DeepSeek API key, entered once per device; never committed to the repository; ?key= links save it then strip themselves

C. Architecture & Key Flows



1. Command lifecycle. Tap mic → recognizer starts (status: "Listening…") → final results accumulate in a buffer → after 1.5 s of silence (or 3.5 s when the phrase ends on a connector such as "by", "the", "search") the full sentence fires → localIntent() strips openers/wake words in a loop and matches open/search/play/close/halt → on a match the action executes immediately and the reply is spoken; otherwise the utterance goes to the LLM (or a local identity/clock answer) → the mic always ends OFF.

2. Real-tab policy. Operations reuse an already-open tab of the same kind (no stacking). If none exists, window.open runs at command time; when a popup blocker catches it, the user gets a one-time guidance message plus an always-working "Open ↵" link, and the next tap anywhere re-opens the blocked URL. No tab is opened at the moment the mic is clicked.

3. Play flow ("play X"). The query is parsed locally (artist extraction with a guard so titles like "stand by me" survive; category queries such as "any comedy video"; "play a song" with no title asks a question and keeps the mic open for the spoken answer, 25 s auto-off). A 9-source client-side parallel race resolves the top YouTube video ID ≈ 2 s; if every source fails the tab still opens on the YouTube results page.

4. Safety rails. Questions are never executed; negations and hypotheticals are ignored; "stop" / "shut up" / "stop Syrax" (including misheard variants like cyrex/psycx/sirex) halt everything instantly and

cut TTS mid-sentence; media is muted while listening so the microphone can never transcribe video audio.

5. Conversation and memory. Chat persists in localStorage (syrax-chat-v1) and is replayed into the model as JSON-shaped turns ({reply, action}) because DeepSeek answers first-try on that format. Identity questions and time/date/world-clock queries are answered locally before the model is called — the LLM cannot know the real time.

6. Deployment. git push to main → GitHub Actions workflow builds with Vite → GitHub Pages publishes hashed assets → live in about two minutes. The repository is public and contains no secrets; the verification suite is run against the deployed URL after every push.

D. Design Decisions, Testing, Limitations

Why a local intent gate instead of pure LLM? Deterministic parsing gives millisecond, testable, hallucination-free execution. An LLM that is allowed to "act" will happily narrate actions it never performed — the exact defect this architecture makes impossible.

Why real browser tabs? Transparency and control: the user sees exactly what the agent did in a normal tab they own, which embedded iframes and sandboxed panels cannot offer (and which matches the project requirement that every operation opens in a real Chrome tab).

Testing strategy Zero-error TypeScript policy; production build verification; an end-to-end suite run against the live site after every deployment covering each documented phrase, wake-word mishearings, popup blocking, resolver failure and keyless operation; console monitored for stray errors.

Known limitations Speech accuracy depends on the browser's recogniser (English-India tuned; noisy environments degrade it). Slow commands can be caught once by the popup blocker until the user selects "Always allow". The LLM requires a user-supplied key for open-ended chat. Video-resolution sources are third-party and may change over time.

Future scope Fish Audio voice clone (JARVIS voice), on-device/offline STT, optional cloud sync with a real database, multi-step chained tasks, always-on assistant widget, more Indian languages, and calendar/email integrations.

E. Viva FAQ — likely questions with model answers

GENERAL

Q. What is Syrax?
A. A voice-driven web agent with a 3D particle interface. You speak one clear sentence; Syrax either performs it in a real browser tab (open, search, play, close) or answers it aloud (identity, time, general knowledge).

Q. What problem does it solve?
A. Routine browser tasks still need hands — menus, typing, tab hunting. Syrax makes them hands-free while keeping every result visible and controllable in a normal tab.

Q. How is it different from Google Assistant or Alexa?
A. Those answer questions and control device-level actions. Syrax operates your actual browser sessions — real tabs you can see and take over — and runs as a transparent, inspectable web app rather than a closed cloud service.

Q. How is it different from ChatGPT voice mode?
A. In Syrax the language model never executes anything; execution comes from a local deterministic parser, so the system cannot claim it opened or played something it did not. Chat-style models converse and may only describe actions.

Q. Why the name “Syrax”?
A. It is our coined assistant name. Because speech engines mishear it (Cyrex, Sirex, Psycx…), the app normalises every known variant in commands, wake words and display.

Q. Who created Syrax?
A. Three students of B.Tech CSE: Ayush Kumar Upadhyay (2500350100012), Navam (2500350100041) and Sandeep (2500350100057).

Q. What were the individual roles?
A. Ayush — architecture, AI engine (intent gate + DeepSeek integration) and voice pipeline; Navam — frontend interface, 3D particle system and PWA installation; Sandeep — browser-operations module, testing/verification and deployment. All three contributed across modules during reviews.

Q. What is the current status?
A. A working, deployed system with the core command set verified live end-to-end; the roadmap covers voice cloning, offline speech and multi-step tasks.

VOICE (STT / TTS / MIC)

Q. Which speech-to-text engine do you use?
A. The browser’s Web Speech API (webkitSpeechRecognition), language set to English (India), running continuously while the mic is on with interim results for feedback.

Q. How do you avoid acting on an incomplete sentence?
A. Results are buffered, not fired immediately. Speech fires only after 1.5 s of silence or, when the phrase ends on a connector (“by”, “the”, “search…), after a 3.5 s grace period — Chrome’s own final-result events also push the flush later.

Q. What is the “one-shot microphone”?
A. The mic switches ON for a single utterance and always returns to OFF after the task. It prevents the assistant hearing its own voice, ambient phantom commands and background listening. Exception: while Syrax is waiting for your answer to its own “which song?” prompt it stays open (25 s auto-off).

Q. How does the mic avoid hearing the video/audio?
A. All media is muted while listening, and the mic is switched off before any reply audio plays, so TTS and video can never loop back as input.

Q. What is the TTS chain?
A. Three layers with timeouts: Fish Audio neural voice (via the optional backend) → Pocket TTS on localhost → the browser’s speechSynthesis, which always works. On static hosting the backend probes are skipped entirely, so there are no console errors.

Q. What happens if STT mishears a word?
A. Known variants of the assistant name are rewritten everywhere (display, commands, halt phrases), common mishearings like “weekend” → “The Weeknd” are mapped, and stuttered repeats (“the the”) are collapsed before parsing.

Q. Does it work offline?
A. Local commands (open/search/play/halt) and identity/clock answers work with no key and no LLM. Internet is needed for the speech service, YouTube results and open-ended chat — and we state that honestly instead of claiming offline operation.

AI / LLM / PRIVACY

Q. Which language model do you use?
A. DeepSeek deepseek-flash with JSON mode, temperature 0.5, max_tokens 384 and thinking disabled — tuned for short, structured, low-latency replies.

Q. Why DeepSeek?
A. Strong quality at very low cost, fast responses, and a simple REST API usable directly from the browser, which keeps the architecture serverless.

Q. What is JSON mode?
A. The model must answer in a fixed structure {reply, action}. It removes parsing ambiguity, makes replies first-try reliable, and lets history be stored in the same shape the model expects.

Q. Can the LLM open a tab or play a video?
A. No — by design it has no actuator. Its system prompt explicitly forbids narrating unperformed actions. Only the local intent gate can trigger operations.

Q. Where is the API key stored?
A. Only in the device's localStorage under syrax-ds-key, pasted once by the user. It is never committed to the repository; ?key= links save it and then remove it from the address bar.

Q. What happens if no key is configured?
A. Every voice command still executes; questions receive canned honest replies and the app offers a one-time key setup panel.

Q. How is memory handled?
A. The conversation is persisted in localStorage (syrax-chat-v1) and re-seeded into the model as the last ~18 JSON-shaped turns after a refresh, so context survives reloads without any server database.

Q. What about privacy?
A. We make only honest claims: prompts are processed by DeepSeek (their provider-side storage applies), the mic is one-shot, media is muted while listening, and we keep no server-side database of our own.

BROWSER OPERATIONS

Q. Why real Chrome tabs instead of an iframe?
A. Real tabs give transparency and full user control, avoid iframe sandbox restrictions and satisfy the project rule that every operation must be visible in the browser.

Q. How do you deal with popup blockers?
A. Tab reuse first; otherwise window.open is invoked at command time; if blocked, the UI gives a one-time "Always allow" guidance plus a permanent "Open ↵" link, and the next tap anywhere retries the blocked URL. Nothing opens at mic-click.

Q. How does "play X" find a video without the YouTube API?
A. A client-side race of nine public sources (invidious instances, proxies, text proxies) returns a valid video ID in about two seconds; the first winner gives a direct watch URL with autoplay.

Q. What if every source fails?
A. The tab still opens on YouTube's search results for the query — the operation degrades instead of hanging.

Q. Does it create duplicate tabs?
A. No — Syrax tracks the tabs it opened by kind and reuses them: a second "search" continues in the already-open search tab.

Q. What does "close the browser" close?
A. Only tabs Syrax opened (tracked by kind); the user's own tabs are never touched.

Q. How do you implement the now-playing strip?
A. Stage state set when a video tab lands and cleared when that tab closes or a new operation replaces it — pure UI reflection of real tab events.

ENGINEERING & DEPLOYMENT

Q. What is the full tech stack?
A. React 19, TypeScript 5.7, Vite 6, Tailwind CSS 4, Three.js + React Three Fiber + GSAP for 3D, Web Speech API for STT/TTS, DeepSeek for NLP, FastAPI (optional sidecar), GitHub Actions + GitHub Pages for CI/CD, PWA manifest/service worker for install.

Q. Why React and Vite?
A. Component model suits the many UI states; Vite gives instant dev start, code-splitting and a tiny production bundle (Three.js is vendor-split from the app code).

Q. Why Three.js particles?
A. They provide continuous state feedback (idle/listening/thinking/speaking plus mic level) as pure client-side rendering — visual identity without extra assets.

Q. Is there a database?
A. No server database. Only browser localStorage (chat + API key). Rationale: static hosting, zero cost, and privacy — nothing about the user leaves their device except prompts sent to the LLM provider.

Q. How is the project deployed?
A. git push to main triggers a GitHub Actions workflow that runs tsc + vite build and publishes to GitHub Pages at ayushupadhyay90.github.io/Syrax — live in ~2 minutes.

Q. How do you test a voice application?
A. Three layers: strict TypeScript (zero errors), production build, and a live end-to-end suite against the deployed URL where every documented phrase is issued (voice simulated) and the resulting tab URL, status line and chat reply are asserted.

Q. Which browsers are supported?
A. Chromium family (Chrome, Edge) on desktop and Android — full Web Speech support. Unsupported browsers get an honest message instead of a broken mic.

Q. Any security measures?
A. No secrets in the repository; API key confined to localStorage; HTTPS hosting; input never executed as code (LLM output is parsed as JSON with a schema, actions whitelisted); questions/negations blocked from executing.

Q. How large is the codebase?
A. About 3,887 lines across 14 frontend source files plus ~310 lines of Python backend.

Q. What is the role of GitHub Actions?
A. Continuous integration and deployment: every push builds the app and redeploys the site, replacing hashed bundles so clients always get the current version.

PROJECT MANAGEMENT

Q. Which methodology did you follow?
A. SDLC in eight phases — requirement analysis, design, UI/3D, voice pipeline, intent+LLM, browser operations, testing, deployment — with a working increment reviewed at the end of each phase.

Q. What was the feasibility conclusion?
A. Fully feasible: browser-native technical stack, near-zero economic cost, minimal training for users, and a schedule that fits the academic timeline.

Q. What are the main limitations?
A. Recogniser accuracy in noise; a one-time popup permission for slow commands; open-ended chat needs the user's own API key; third-party video sources can change; iOS not yet verified.

Q. What is the future scope?
A. Voice cloning (Fish Audio), offline/on-device STT, optional cloud sync with a real database, multi-step tasks, always-on widget, more languages, and integrations such as calendar and email.

Q. What did the team learn?
A. Layered design separating deterministic actions from probabilistic language; browser security constraints (popup policies); graceful fallback engineering; and verification-driven development against a live deployment.

Q. How do you handle errors and failures?
A. Every external dependency has a fallback ladder: TTS falls back to the browser voice, video resolution falls back to the results page, blocked popups fall back to guidance plus a link, and a missing key falls back to canned replies. Nothing can hang.

Tip: for the live demo keep the order — open → search → play (with artist) → question → stop. It exercises every subsystem in under a minute.